

**AI Lab-1**  
**BS DS FALL 2024 Afternoon**

**Introduction to 15-Puzzle:**

The 15-puzzle consists of:

- A 4×4 grid
- 15 numbered tiles (1–15)
- One blank tile (0)



**Checking Solvability**

Before solving the puzzle, we must first check whether the configuration is solvable. Only half of all  $N \times N$  puzzle states are reachable. If a puzzle is unsolvable, the search algorithm will run indefinitely or waste computation.

**Inversion Count (I):**

Flatten the grid row-wise (ignore 0). Count pairs  $(i, j)$  such that  $i$  appears before  $j$  and  $i > j$ .

**Blank Row (R):**

Count the row of the blank tile from the bottom (bottom row = 1).

**General Solvability Condition (N-Puzzle)**

Let:

- **I** = number of inversions
- **R** = row of the blank tile counted from the bottom
- **n** = grid size

If **n** is odd:

$$I \bmod 2 = 0$$

If **n** is even:

$$(I + R) \bmod 2 = 1$$

If the condition holds, the puzzle is solvable. Otherwise, it is not solvable.

## TASK 01: 15-puzzle through Iterative Deepening Depth-First Search (IDDFS)

Implement Iterative Deepening Depth-First Search (IDDFS) to solve the 15-puzzle problem.

### Code Template:

```
class Puzzle:
    def __init__(self, initial_state, goal_state):
        self.initial_state = initial_state
        self.goal_state = goal_state
    def depth_limited_search(self, state, depth, visited):
        """Performs Depth-Limited Search (DLS) up to a given depth."""
        Pass
    def iddfs(self):
        """Implement Iterative Deepening Depth-First Search to solve the puzzle."""
        Pass
    def generate_moves(self, state):
        """Generate valid moves from the current state."""
        Pass
    def is_solvable(self, state):
        """Check if the given state is solvable."""
        Pass

def main():
    initial_state = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 0, 15]]
    goal_state = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 0]]

    puzzle = Puzzle(initial_state, goal_state)
    print("IDDFS Solution:", puzzle.iddfs())
```

### Lab Tasks:

1. Implement **Iterative Deepening Depth-First Search (IDDFS)** to solve the 15-Puzzle problem using the given code template.
2. Implement the **solvability check** before running the search algorithm.
3. Test the program on the provided initial configuration.
4. Additionally, test at least **two different initial configurations** (one solvable and one unsolvable).
5. For each case, display whether the puzzle is solvable and print the solution path if found.

## Task 02: 15-Puzzle through A\* Algorithm

### Code template:

```
import time

class PuzzleNode:
    def __init__(self, state, parent, move, g_cost, h_cost):
        """Initialize the puzzle node"""
        self.state = state
        self.parent = parent
        self.move = move
        self.g_cost = g_cost
        self.h_cost = h_cost
        self.f_cost = g_cost + h_cost

    def generate_children(self):
        """Generate new states by moving the empty tile"""
        pass

    @staticmethod
    def calculate_heuristic(state, goal_state, heuristic_type="manhattan"):
        """Calculate Manhattan Distance, Euclidean Distance, Misplaced Tiles
and any 2 other heuristic"""
        pass

    def __lt__(self, other):
        """ Ensures nodes are compared by f_cost in heapq. """
        return self.f_cost < other.f_cost

class PuzzleSolver:
    def __init__(self, start_state, goal_state):
        """Initialize with start and goal state"""
        self.start_state = start_state
        self.goal_state = goal_state

    def is_solvable(self, state):
        """ Check if the puzzle configuration is solvable."""
        pass

    def astar_search(self, heuristic_type="manhattan"):
        """Implement A* algorithm"""
        pass

    def trace_solution(self, node):
        """Trace back the solution path"""
        pass
```

## Sample Run:

```
# Sample initial and goal states

initial_state = [[1, 2, 3, 4], [5, 6, 7, 8],[9, 10, 11, 12],[13,14, 0, 15]]
goal_state = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13,14, 15, 0]]

# Create PuzzleSolver object
solver = PuzzleSolver(initial_state, goal_state)

# Check if the puzzle is solvable
if solver.is_solvable(initial_state):
    print("The puzzle is solvable. Proceeding with A* Search...\n")

    # Run A* Search with Manhattan Distance heuristic
    print("Running A* Search (Manhattan Distance)...")

    start_time = time.time()
    astar_solution_manhattan = solver.astar_search(heuristic_type="manhattan")
    end_time = time.time()

    if astar_solution_manhattan:
        print("\nSolution (Manhattan Distance): ")
        solver.trace_solution(astar_solution_manhattan)
        print(f"Execution Time: {end_time - start_time:.5f} seconds\n")
    # Run A* Search with Misplaced Tiles heuristic
    print("Running A* Search (Misplaced Tiles)...")

    start_time = time.time()
    Astar_solution_misplaced = solver.astar_search(heuristic_type="misplaced_tiles")
    end_time = time.time()

    if astar_solution_misplaced:
        print("\nSolution (Misplaced Tiles): ")
        solver.trace_solution(astar_solution_misplaced)
        print(f"Execution Time: {end_time - start_time:.5f} seconds\n")
else:
    print("The puzzle is NOT solvable.")
```

## Lab Tasks

1. Develop an A\* algorithm to solve the 15-puzzle problem.
2. Implement a heuristic-based approach using Manhattan Distance, Euclidean Distance, Misplaced Tiles and any 2 other heuristics for the puzzle.
3. Check if the puzzle configuration is solvable before attempting a solution.
4. Compare the performance of exhaustive search (IDDFS) and of all 5 heuristic methods in solving the 15-puzzle problem by measuring execution time and counting the number of nodes expanded.